

Cora — Seguridad y Buenas Prácticas

Prácticas implementadas y modelo de 3 roles (Admin, Usuario, Auditor)

Este documento distingue explícitamente entre lo que **ya está implementado y verificado** en el proyecto y lo que es un **modelo propuesto** para el ítem de la rúbrica (3 roles). Cora es hoy una app de consumo mono-usuaria; no tiene panel de administración. Documentamos esa realidad honestamente y proponemos cómo extenderla.
Repositorio: github.com/eduardoevz/Volcanic-2026

1. Buenas prácticas ya implementadas

- **RLS en las 26 tablas** de la base de datos, auditado en `docs/RLS_AUDIT.md`. Cada política usa `auth.uid() = user_id`.
- **Cero secretos hardcoded en el cliente**: `service_role` y API key de Gemini solo en Edge Functions server-side.
- **Sesión cifrada localmente** con AES-256 vía `expo-secure-store`, nunca tokens en texto plano.
- **PKCE** en login, Google y reset de contraseña.
- **Consentimiento explícito y revocable** antes de compartir datos con la IA (apagado por defecto).
- **Círculo familiar granular**: sin invitación aceptada, cero acceso — vía RPCs que solo devuelven señales agregadas.
- **Datos públicos gateados fila por fila** (`specialists.consent_to_publish`).
- **Sin inyección SQL**: query builder / RPCs parametrizados en todo el acceso a datos.
- **Código legible y convencional**: TypeScript estricto, ESLint, un módulo por feature.

2. Estado actual de roles (honesto)

Hoy el proyecto tiene, en la práctica, **un solo rol de aplicación**: Usuario — cualquier persona autenticada, con acceso exclusivo a sus propios datos vía RLS. No existe una tabla `user_roles`, panel de administración, ni cuenta de auditor en el código. El equivalente más cercano a un rol elevado es la `service_role` key que usan las Edge Functions internas (nunca expuesta al cliente).

3. Modelo propuesto de 3 roles y permisos

Extensión propuesta sin romper el aislamiento actual: columna `app_role` en `profiles` y políticas RLS adicionales vía función `security_definer` (mismo patrón ya usado para el círculo familiar).

ROL	QUIÉN	PERMISOS	NO PUEDE
Admin	Equipo Cora (hoy solo vía <code>service_role</code>)	Gestionar catálogo de contenido y directorio de salud. Solo tablas de catálogo/contenido.	Leer <code>daily_logs</code> , <code>cycles</code> , <code>pregnancies</code> , <code>medical_background</code> ni ninguna tabla con <code>user_id</code> de usuaria.
Usuario (implementado)	Cualquier persona registrada	Lectura/escritura exclusiva de sus propias filas (<code>user_id = auth.uid()</code>).	Leer/modificar datos de otra usuaria, incluso en su círculo familiar.

ROL	QUIÉN	PERMISOS	NO PUEDE
Auditor	Seguridad/cumplimiento externo	Solo lectura de metadatos: políticas RLS activas, logs, estructura — nunca contenido clínico.	Escribir cualquier tabla. Leer notas, mensajes de IA o antecedentes médicos.

Boceto de políticas RLS

```
-- Columna de rol, por defecto 'usuario' (no rompe el modelo actual)
alter table profiles add column app_role text not null default 'usuario'
  check (app_role in ('usuario', 'admin', 'auditor'));

-- Función security definer para leer el rol sin recursión de RLS
create or replace function public.current_app_role()
returns text language sql security definer stable as $$
  select app_role from profiles where id = auth.uid();
$$;

-- Admin: puede escribir el catálogo de contenido, nunca datos clínicos
create policy "admin_manage_content" on educational_content
  for all using (current_app_role() = 'admin');

-- Auditor: solo lectura de políticas activas, nunca de datos de usuarias
create policy "auditor_read_policies" on pg_policies
  for select using (current_app_role() = 'auditor');

-- Usuario (ya vigente en las 26 tablas, sin cambios):
create policy "own_rows_only" on daily_logs
  for all using (auth.uid() = user_id);
```

4. Por qué este diseño

El principio guía es el mismo que ya rige el círculo familiar y la IA en el código real: ningún rol elevado gana acceso a datos clínicos personales por defecto. Un Admin de catálogo no necesita ver el diario de síntomas de nadie, y un Auditor de seguridad necesita ver que las políticas existen y funcionan, no el contenido que protegen. Separar "administrar contenido" de "leer salud personal" evita que un solo rol comprometido exponga toda la base de usuarias.